Multi-Omics Integration Analysis - DIABLO

Adam El Idrissi

2026-07-20

Contents

1. Practical overview **2**

2. Package installation and loading **2**

3. Load expression and methylation data **2**

4. Load metadata and align sample identifiers **3**

5. Normalize the RNA-seq data **5**

6. Filter and transform the methylation data **5**

7. Prepare the data blocks for DIABLO **6**

8. Fit the first DIABLO model **7**

9. Tune the number of components **8**

10. Downstream analysis **10**

10.1 Sample projections **10**

10.2 Methylation and RNA loading plot **10**

10.3 Extract selected variables **12**

10.4 Correlation-circle variable plot **15**

10.5 Generate the DIABLO clustered image map **16**

10.6 Prepare heatmap annotations **17**

10.7 Sample-arrow plot **19**

10.8 Circos plot **20**

10.9.a Extract high-correlation RNA–methylation pairs **21**

10.9.b Correlation heatmap **22**

10.10 RNA–methylation correlation network **24**

11. Performance evaluation **25**

12. Optional top-feature network analysis **30**

13. Acknowledgement **31**

1. Practical overview

This R Markdown document preserves the workflow and execution order of the supplied `MixOmics.R` script. This markdown contains all the detailed description regarding the code cells and the output results, relatively to the data subset utilized. It is worth taking a look on the process of the data integration using DIABLO approach, with its alternative tutorial being explained in `https://mixomics.org/mixdiablo/diablo-tcga-case-study/`. We have approached this data fusion framework "DIABLO" as one of the most utilized supervised approach for data integration.

---

2. Package installation and loading

Packages are installed from both CRAN and Bioconductor repositories. `mixOmics` is the main package used for multi-omics integration, as it supports the integration of multiple omics datasets through methods such as `DIABLO`, which will be used throughout this tutorial. `BiocParallel` is used for parallel computing, allowing the analysis to run faster, particularly during model tuning and performance evaluation, which can require considerable computation time. It is generally recommended to allocate approximately (the total number of available CPU cores − 2) workers to maximize performance while leaving enough resources for the operating system. `tidyverse`, `dplyr`, and their associated packages are used for data manipulation and dataframe handling. Finally, `reshape2`, `edgeR`, and `limma` provide functions for processing and preparing the RNA-seq and methylation datasets prior to multi-omics integration.

Before starting with the tutorial, please ensure that the packages are installed, by executing the R script file: `diablo_R_packages_to_install.R`.

```
######## MUlti Omics Integration analysis ########################################
#BiocManager::install("mixOmics")
library(BiocParallel)
library(mixOmics)
library(tidyverse)
library(GEOquery)
library(dplyr)
library(pheatmap)
library(caret)
library(RColorBrewer)
library(reshape2)
library(igraph)
library(edgeR)
library(limma)

cores <- MulticoreParam(workers = 22)
```

---

3. Load expression and methylation data

Boar taint is one of the most extensively studied phenotypes in swine production. It is characterized by an unpleasant urine- or feces-like odor and flavour that is released during the cooking of meat from uncastrated

male pigs. Based on the intensity of this phenotype, the animals included in this study were classified into three groups: Low, Medium, and High boar taint.

Our two datasets originate from the 3 groups of boars, differentiated by their taints (Low - Medium - High taint), where one dataset contains Testies RNA-seq measurements and the other contains Methylation measurements. The RNA matrix consists of 25,322 genes measured across 9 boars (3 from each phenotypes), whereas the methylation matrix contains 100,000 methylation observations measured from the same 9 boars (3 from each phenotypes). For vertical integration using the DIABLO framework, it is essential that the same biological samples are shared across all omics modalities. In our case, the nine boars were profiled at different molecular levels, allowing the integration of transcriptomic and methylation information from the same individuals. The data are organized in the required format, where rows correspond to molecular features (genes or methylation sites) and columns correspond to the individual samples. Later in the workflow, these matrices are transposed into the format required by DIABLO, where rows represent the individual observations (samples) and columns represent the variables (molecular features).

```
#### Load the Expression data nd methylation data
rna_matrix=read.csv("RNA_Seq_Testis.csv")
rna_matrix$X=NULL
rownames(rna_matrix)=rna_matrix$GeneID
rna_matrix$GeneID=NULL
head(rna_matrix)
```

```
##                     X7107 X9138 X1306 X8308 X1113 X2111 X1137 X9312 X1105
## ENSSSCG00000000001     6     2     6    49     4     6     4     1     0
## ENSSSCG00000000002    90    56    29    19    23    54    36    56    13
## ENSSSCG00000000003   154   129   114   178   102   118   109   104    59
## ENSSSCG00000000004  6718  8013  4429   178  7456  6912 10963  7796  3389
## ENSSSCG00000000005    57    76    31    57    39    45    63   105    28
## ENSSSCG00000000006    33    62    17     3    20    33    50     4    13
```

```
meth_matrix_filtered =read.csv("Epigenetic_matrix_filtered_100000.csv")
rownames(meth_matrix_filtered)=meth_matrix_filtered$region
meth_matrix_filtered$X=NULL
meth_matrix_filtered$X.1=NULL
meth_matrix_filtered$region=NULL
head(meth_matrix_filtered)
```

```
##                   X1105     X1137     X9312      X1113 X2111     X8308     X1306     X7107     X9138
## chr4_1126216  0.8750000 0.7195122 0.9629630 0.59677419  0.75 0.7884615 0.7894737 0.6410256 0.5555556
## chr18_2996989 0.0000000 0.4418605 0.0000000 1.00000000  0.00 0.3571429 0.3500000 0.0000000 0.3333333
## chr2_513273   0.0000000 0.0000000 0.0000000 0.00000000  0.00 0.0000000 0.0000000 0.0000000 0.0000000
## chr5_1441285  0.8888889 0.6796117 0.7857143 0.89583333  0.50 0.8888889 0.7800000 0.7173913 0.7500000
## chr3_44050407 0.0000000 1.0000000 0.0000000 0.00000000  1.00 0.0000000 1.0000000 1.0000000 1.0000000
## chr3_5000684  0.0000000 0.0000000 0.0000000 0.08333333  0.00 0.0000000 0.0000000 0.0000000 0.0000000
```

---

4. Load metadata and align sample identifiers

The metadata file contains the identifiers of each boar together with its corresponding boar taint phenotype (Low, Medium, or High). In this step, the metadata are prepared to be compatible with the DIABLO integration framework. One important concept during multi-omics integration is `column alignment`. This means that the order of the samples must be identical across every omics dataset to be integrated. For example, if Dataset A contains the sample order ID1, ID2, ID3, then Dataset B and Dataset C must follow the exact same order (ID1, ID2, ID3). Any mismatch in the sample ordering would result in the integration of measurements would result potential results biasm as the matrices do not observe the row.names and

col.names information.

```
## Loading Metadata and Filtering ################################################

metadat_columns =read.csv("metadata.csv")
head(metadat_columns)
```

```
##      X SampleID           Group BoarTaint
## 1 1326     1326 high boar taint         3
## 2 7106     7106 high boar taint         3
## 3 1331     1331 high boar taint         3
## 4 1334     1334 high boar taint         3
## 5 1338     1338 high boar taint         3
## 6 7107     7107 high boar taint         3
```

```
metadat_columns$X=NULL
metadat_columns=data.frame(metadat_columns)

# Add prefix based on boar taint group
metadat_columns$SampleName <- ifelse(
  grepl("high", metadat_columns$Group, ignore.case = TRUE),
  paste0("H", metadat_columns$SampleID),
  ifelse(
    grepl("medium", metadat_columns$Group, ignore.case = TRUE),
    paste0("M", metadat_columns$SampleID),
    ifelse(
      grepl("low", metadat_columns$Group, ignore.case = TRUE),
      paste0("L", metadat_columns$SampleID),
      paste0("X", metadat_columns$SampleID)  # fallback if none match
    )
  )
)

metadat_columns$Group <- sub(" .*", "", metadat_columns$Group)

# Create the old and new column name mapping
metadat_columns$OldSampleName <- paste0("X", metadat_columns$SampleID)  # Original names
name_map <- setNames(metadat_columns$SampleName, metadat_columns$OldSampleName)  # Named vector: old ->

# Rename methylation matrix columns
common_meth_cols <- intersect(colnames(meth_matrix_filtered), names(name_map))
colnames(meth_matrix_filtered)[match(common_meth_cols, colnames(meth_matrix_filtered))] <- name_map[com

# Rename RNA-seq matrix columns
common_rna_cols <- intersect(colnames(rna_matrix), names(name_map))
colnames(rna_matrix)[match(common_rna_cols, colnames(rna_matrix))] <- name_map[common_rna_cols]

# Update metadata rownames to match new names
rownames(metadat_columns) <- metadat_columns$SampleName


# Filter metadata to keep only samples present in omics data
omics_samples <- colnames(meth_matrix_filtered)
sample_metadata <- metadat_columns[metadat_columns$SampleName %in% omics_samples, ]
```

```r
# Double-check that sample orders match
all(colnames(rna_matrix) == colnames(meth_matrix_filtered))
```

```
## [1] FALSE
```

```r
# Reorder columns of methylation matrix to match RNA-seq
meth_matrix <- meth_matrix_filtered[, colnames(rna_matrix)]
####### Check that they're now aligned
all(colnames(meth_matrix) == colnames(rna_matrix))  # should return TRUE
```

```
## [1] TRUE
```

---

5. Normalize the RNA-seq data

Before integrating the transcriptomic data with any other data modality, the RNA-seq counts must be normalized to create a stable distribution. The raw count matrix are initially pre-processed by removing lowly expressed genes using `filterByExpr()`. Normalization factors are then estimated with `calcNormFactors()` for batch to batch estimation and false positive/negative results, and the counts are transformed into log2 Counts Per Million (CPM) values using a prior count to avoid undefined values produced by log(0) (mathematically undefined real number). The final processed RNA-seq data leaves with 9 boars and 10546

```r
###### Normalize the data
dge <- DGEList(counts = rna_matrix)
keep <- filterByExpr(dge)
```

```
## Warning in filterByExpr.DGEList(dge): All samples appear to belong to the same group.
```

```r
dge <- dge[keep, , keep.lib.sizes = FALSE]
dge <- calcNormFactors(dge)
# Log2 CPM transformation with prior count to avoid log(0)
rna_matrix <- cpm(dge, log = TRUE, prior.count = 1)
dim(rna_matrix)
```

```
## [1] 10546     9
```

---

6. Filter and transform the methylation data

Before integrating the methylation data with any other data modalities, in the same process of RNA-seq, methylation data is subject to pre-processing. The data are first converted into a numeric matrix, after which the methylation -values are transformed into M-values using a logit transformatiion. This transformation is presented as log2 ratio transformation of the intensities methylated probes vs unmethylated probes. The reference of this transformation is in the following paper `https://pmc.ncbi.nlm.nih.gov/articles/PMC3012676/`. To improve data quality, methylation sites with more than 20% missing values are removed, and samples exceeding the same missingness threshold are also excluded. The final outcome of the methylation dataset is 18675 methylated regions in 9 candidate boars.

```r
####### Filter Methylation data
# Convert data frame to numeric matrix
meth_matrix <- as.matrix(meth_matrix)
storage.mode(meth_matrix) <- "numeric"  # make sure all values are numeric
# Convert to M-values
meth_matrix <- log2(meth_matrix / (1 - meth_matrix))
```

```r
meth_matrix[is.infinite(meth_matrix)] <- NA
# keep features with <20% NA
meth_matrix <- meth_matrix[apply(meth_matrix, 1, function(x) sum(is.na(x)) < length(x) * 0.2), ]
meth_matrix <- meth_matrix[, colSums(is.na(meth_matrix)) < nrow(meth_matrix) * 0.2]

dim(meth_matrix)
```

```
## [1] 18675     9
```

---

7. Prepare the data blocks for DIABLO

Before fitting the DIABLO model, both omics datasets must be organized into the format required by the integration framework. The phenotypic information is first extracted from the metadata and converted into a categorical response variable (Y), representing the three boar taint groups (Low, Medium, and High). The normalized RNA-seq and filtered methylation datasets are then combined into a list of omics blocks, where each block corresponds to one molecular modality.

A design matrix is subsequently defined to specify the relationships between the omics datasets during integration. In this study, a full design is adopted, allowing both RNA-seq and methylation data to contribute equally to the construction of the latent components. For a given example of 3 data modalities, the value of equal contribution will be set to 0.5.

The number of latent components (ncomp) is initially set to five, while the number of variables retained from each omics block (keepX) is fixed at 50 features per component. These parameters provide an initial model that will later be optimized through cross-validation. Finally, both omics matrices are transposed into the format required by DIABLO, where rows correspond to biological samples and columns correspond to omics variables.

```r
# Set sample group (outcome variable) from metadata
Y <- as.factor(sample_metadata$Group)
names(Y) <- sample_metadata$SampleName

# Combine omics into a list
omics_list <- list(
  RNA = rna_matrix,
  Meth = meth_matrix
)

# Check dimensions
lapply(omics_list, dim)
```

```
## $RNA
## [1] 10546     9
##
## $Meth
## [1] 18675     9
```

```r
# Full design = all data blocks contribute to the response
design <- matrix(1, ncol = length(omics_list), nrow = length(omics_list))
diag(design) <- 0
colnames(design) <- rownames(design) <- names(omics_list)
design
```

```
##      RNA Meth
```

```
## RNA    0    1
## Meth   1    0
```

```
# Set number of components
ncomp <- 5


# Set number of variables to keep from each omics block per component
list.keepX <- list(
  RNA = rep(50, ncomp),       # top 50 genes per component
  Meth = rep(50, ncomp)       # top 50 CpG per component
)

# Convert omics_list to proper DIABLO format
omics_list_fixed <- lapply(omics_list, function(df) {
  mat <- as.matrix(df)
  mat <- t(mat)                   # Transpose: samples as rows, features as columns
  storage.mode(mat) <- "numeric" # Ensure matrix is numeric
  return(mat)
})


########################
# Check shape
str(omics_list_fixed)
```

```
## List of 2
##  $ RNA : num [1:9, 1:10546] 3.28 2.49 2.14 1.29 1.8 ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : chr [1:9] "H7107" "H9138" "H1306" "M8308" ...
##   .. ..$ : chr [1:10546] "ENSSSCG00000000002" "ENSSSCG00000000003" "ENSSSCG00000000004" "ENSSSCG0000
##  $ Meth: num [1:9, 1:18675] 0.837 0.322 1.907 1.898 0.566 ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : chr [1:9] "H7107" "H9138" "H1306" "M8308" ...
##   .. ..$ : chr [1:18675] "chr4_1126216" "chr5_1441285" "chrUn_NW_018085262v1_40801" "chr12_1327113"
```

---

8. Fit the first DIABLO model

This step establishes the first DIABLO model by integrating the two omics datasets with the boar taint phenotype. The model is initially built using five latent components, with the top 50 variables retained from each omics block for every component. The previously defined full design matrix is used, allowing both omics datasets to contribute equally to the construction of the latent components. This initial model serves as the foundation for the subsequent tuning, performance evaluation, and downstream analyses.

```
# Run DIABLO
diablo_model <- block.splsda(
  X = omics_list_fixed,
  Y = Y,
  ncomp = ncomp,
  keepX = list.keepX,
  design = design
)
```

```
## Design matrix has changed to include Y; each block will be
```

```
##             linked to Y.
```

---

9. Tune the number of components

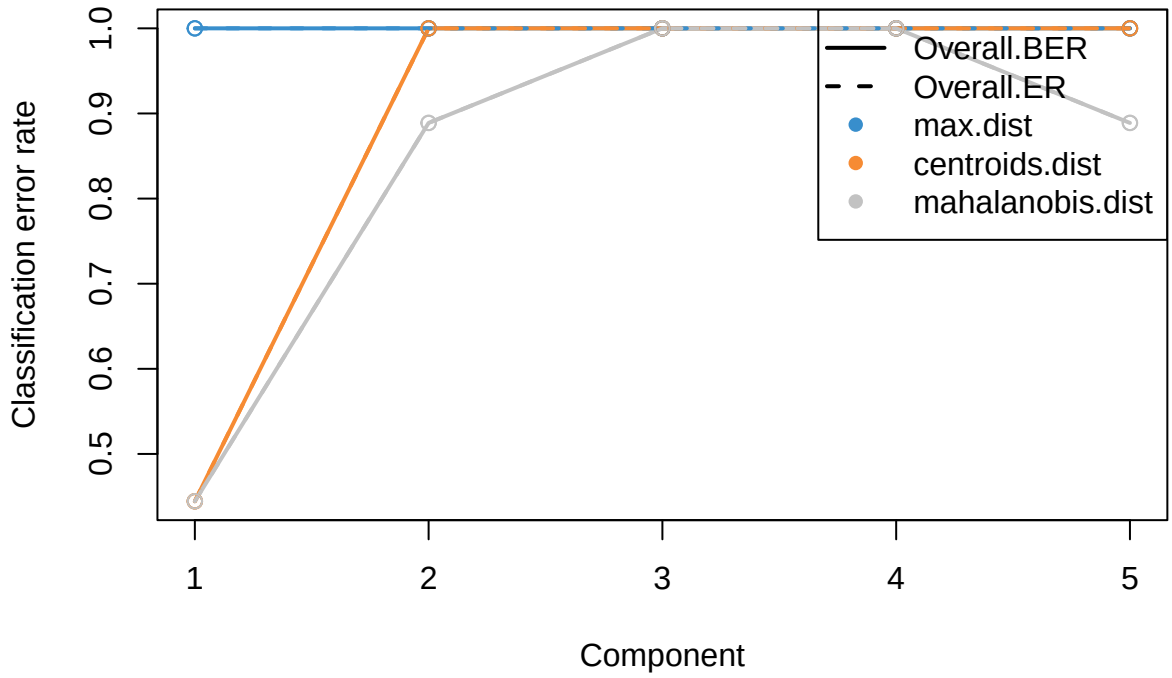
The initial DIABLO model is subsequently evaluated to determine the optimal number of latent components required to discriminate between the boar taint phenotypes. This optimization is performed using Leave-One-Out validation method, as our data contains very few observation, repeated 10 times. DIABLO supports the paralellization frameworkm to allow the previously defined CPU cores to accelerate the repeated validation method. Note that, when dealing with abundant observations, consider using the `Mfold` validation approach, and include the number of folds using the argument `folds = number_of_folds`. (3,5,10 folds) Given the limitation of the current dataset, the optimal approach is to set the validation using the jacknife method `loo`.

The resulting performance plot illustrates how the prediction error changes as the number of components increases. Note that our datasets have limitation in terms of observation, which the models does fail at minimizing the error from `comp 2 and higher`. The best metric to evaluate is the centroids.dist, in order to keep the most significant informative components throughout the model tuning. Finally, the optimal number of components (ncomp) is selected as the one producing the lowest Overall Balanced Error Rate according to the Weighted Vote classification using the centroids distance metric. As the value is set to `ncomp = 1`, we will choose the first two components to finalize the DIABLO model.

```
# run component number tuning with repeated CV
perf.diablo = perf(diablo_model, validation = 'loo',
                   nrepeat = 10, BPPARAM = cores, progressBar = TRUE)
```

```
##
## Performing repeated cross-validation with nrepeat = 10...
##   |
##   |
##   |
##   |
##   |
##   |
##   |
##   |
##   |
##   |
##   |
```

```
plot(perf.diablo) # plot output of tuning
```




```
# set the optimal ncomp value
ncomp = perf.diablo$choice.ncomp$WeightedVote["Overall.BER", "centroids.dist"] +1 # Reason is due to on
# show the optimal choice for ncomp for each dist metric
perf.diablo$choice.ncomp$WeightedVote
```

```
##             max.dist centroids.dist mahalanobis.dist
## Overall.ER         1              1                1
## Overall.BER        1              1                1
```

```
perf.diablo$choice.ncomp$MajorityVote
```

```
##             max.dist centroids.dist mahalanobis.dist
## Overall.ER         1              1                1
## Overall.BER        1              1                1
```

```
list.keepX <- list(
  RNA = rep(50, ncomp),       # top 50 genes per component
  Meth = rep(50, ncomp)       # top 50 CpG per component
)

diablo_model = block.splsda(X = omics_list_fixed, Y = Y, ncomp = ncomp,
                            keepX = list.keepX, design = design)
```

```
## Design matrix has changed to include Y; each block will be
##             linked to Y.
```

---

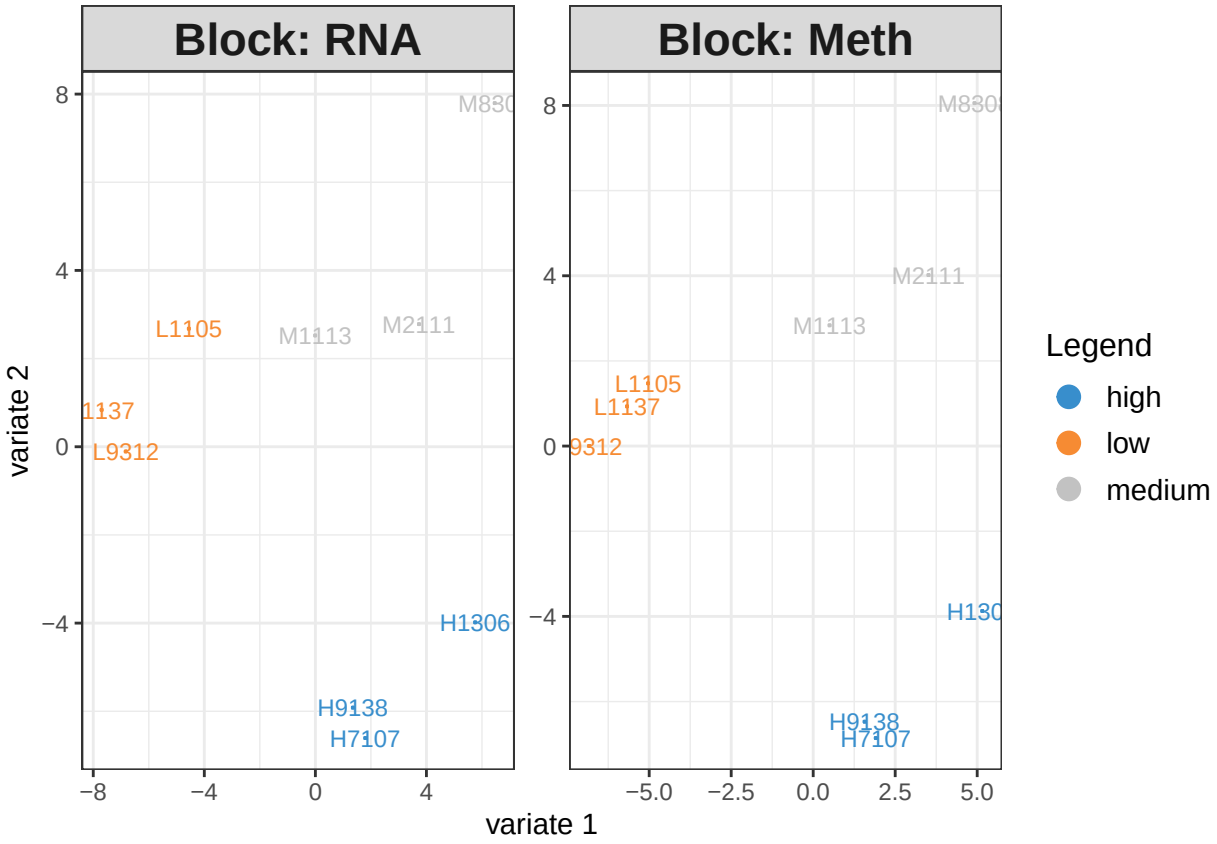
10. Downstream analysis

10.1 Sample projections

Throughout the tuning procedure, we selected two components, then we plotted the individuals to observe how they cluster according to their omics measurements. As observed, both the RNA-seq and methylation plots stratify the subjects into three distinct regions. In both datasets, Component 1 separates the Medium boar taint group from the Low and High groups. As the second component was not a recommendation from the model, it is better to leave the interpretation aside.

```
############ Downstream analysis ###################################################

# Plot sample projections on the first two components from DIABLO
plotIndiv(diablo_model, group = Y, legend = TRUE,alpha = 1)
```



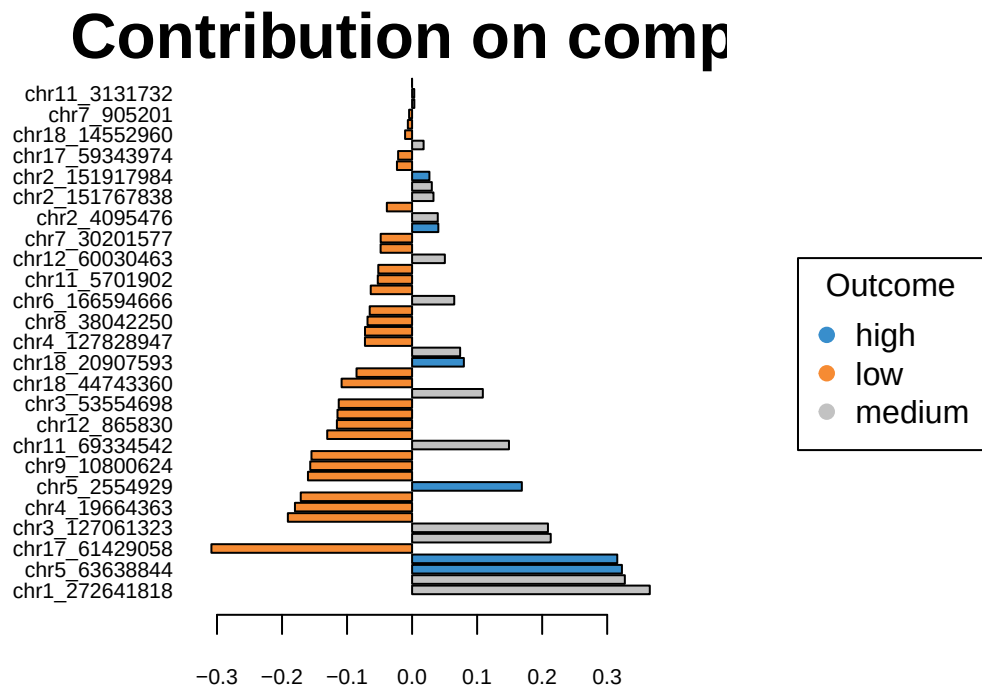

```
# Uncomment this block if you're running old linux OS
# ragg::agg_png(filename = "plot_indiv.png", width = 2400, height = 1400, units = "px", res = 300, back
# plotIndiv(diablo_model, group = Y, legend = TRUE,alpha = 1)
# dev.off()
```

10.2 Methylation and RNA loading plot

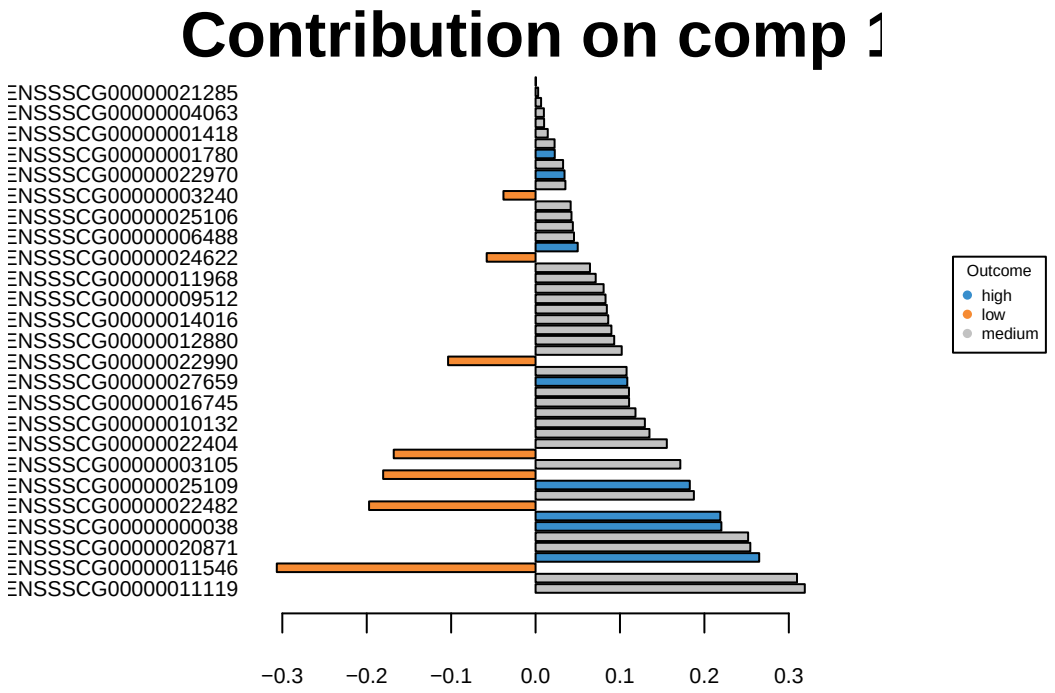
We have plotted the variables with the highest contribution from the methylation dataset, where each variable contributes to the differentiation of the sample groups along latent component 1. In other words, each bar represents the contribution of a methylation feature, while its colour indicates the phenotype that primarily

drives its discriminatory effect. The same interpretation applies to the RNA-seq modality, where the highest contributing genes are displayed according to their association with the different boar taint phenotypes

```
# Features with maximum contribution are highlighted using the 'median' method
plotLoadings(diablo_model, comp = 1, block = "Meth", contrib = 'max', method = 'median', size.legend = 1
```




```
plotLoadings(diablo_model, comp = 1, block = "RNA", contrib = 'max', method = 'median', size.legend = 0
```




```
# Uncomment this block if you're running old linux OS
# ragg::agg_png(filename = "plot_loading_methylation.png", width = 3200, height = 1800, units = "px", r
# plotLoadings(diablo_model, comp = 1, block = "Meth", contrib = 'max', method = 'median', size.legend =
# dev.off()
#
# ragg::agg_png(filename = "plot_loading_RNA.png", width = 3200, height = 1800, units = "px", res = 300
# plotLoadings(diablo_model, comp = 1, block = "RNA", contrib = 'max', method = 'median', size.legend =
# dev.off()
```

10.3 Extract selected variables

To extract the most significant biomarkers observed for each matrix block, we use `selectvar()` to extract the significance of each variable, from the final DIABLO model.

```
# This gives a vector of methylation features selected by DIABLO - TOP 50
selectVar(diablo_model, block = "Meth", comp = 1) -> var1; var1
```

```
## $Meth
## $Meth$name
##  [1] "chr1_272641818"          "chr18_44814773"          "chr5_63638844"           "cl
##  [5] "chr17_61429058"          "chr9_10768294"           "chr3_127061323"          "cl
##  [9] "chr4_19664363"           "chr17_35677192"          "chr5_2554929"            "cl
## [13] "chr9_10800624"           "chr16_3566139"           "chr11_69334542"          "cl
## [17] "chr12_865830"            "chr2_150081780"          "chr3_53554698"           "cl
## [21] "chr18_44743360"          "chr3_129945605"          "chr18_20907593"          "cl
```

```
## [25] "chr4_127828947"              "chr18_49057420"               "chr8_38042250"               "cl
## [29] "chr6_166594666"              "chrUn_NW_018084979v1_2705218" "chr11_5701902"               "cl
## [33] "chr12_60030463"              "chr6_159053377"               "chr7_30201577"               "cl
## [37] "chr2_4095476"                "chrUn_NW_018084866v1_149233"  "chr2_151767838"              "cl
## [41] "chr2_151917984"              "chr6_169681357"               "chr17_59343974"              "cl
## [45] "chr18_14552960"              "chr11_6125282"                "chr7_905201"                 "cl
## [49] "chr11_3131732"               "chr13_68986749"
##
## $Meth$value
##                                    value.var
## chr1_272641818                  0.3655167847
## chr18_44814773                  0.3271362410
## chr5_63638844                   0.3226718297
## chr8_1205413                    0.3155122445
## chr17_61429058                 -0.3086096931
## chr9_10768294                   0.2131931665
## chr3_127061323                  0.2089076048
## chr2_10192905                  -0.1910235556
## chr4_19664363                  -0.1801179418
## chr17_35677192                 -0.1712493702
## chr5_2554929                    0.1687622680
## chr3_41556385                  -0.1601335651
## chr9_10800624                  -0.1566369532
## chr16_3566139                  -0.1547421770
## chr11_69334542                  0.1489589267
## chr4_15436578                  -0.1305340023
## chr12_865830                   -0.1156348232
## chr2_150081780                 -0.1147334700
## chr3_53554698                  -0.1127927459
## chr8_351279                     0.1088059185
## chr18_44743360                 -0.1084259568
## chr3_129945605                 -0.0854508742
## chr18_20907593                  0.0796294781
## chr14_135390401                 0.0738785367
## chr4_127828947                 -0.0724976507
## chr18_49057420                 -0.0724141224
## chr8_38042250                  -0.0685583381
## chr1_9524920                   -0.0651982410
## chr6_166594666                  0.0648307438
## chrUn_NW_018084979v1_2705218 -0.0635853759
## chr11_5701902                  -0.0527928369
## chr2_140880739                 -0.0519901675
## chr12_60030463                  0.0504079077
## chr6_159053377                 -0.0484197304
## chr7_30201577                  -0.0483188694
## chrUn_NW_018084866v1_215415    0.0403555532
## chr2_4095476                    0.0394960088
## chrUn_NW_018084866v1_149233   -0.0388443147
## chr2_151767838                  0.0328667089
## chr13_199635186                 0.0303560840
## chr2_151917984                  0.0266649949
## chr6_169681357                 -0.0232985006
## chr17_59343974                 -0.0214769881
## chr3_132356259                  0.0177084721
```

```
## chr18_14552960               -0.0107682729
## chr11_6125282                -0.0065004381
## chr7_905201                  -0.0045864905
## chr3_118286711                0.0034568425
## chr11_3131732                 0.0033210852
## chr13_68986749               -0.0001096732
##
##
## $comp
## [1] 1
```

```
dim(var1$Meth$value)
```

```
## [1] 50  1
```

```
selectVar(diablo_model, block = "RNA", comp = 1) -> var1; var1
```

```
## $RNA
## $RNA$name
##  [1] "ENSSSCG00000011119" "ENSSSCG00000026418" "ENSSSCG00000011546" "ENSSSCG00000017092" "ENSSSCG0000
##  [7] "ENSSSCG00000000038" "ENSSSCG00000024536" "ENSSSCG00000022482" "ENSSSCG00000021418" "ENSSSCG0000
## [13] "ENSSSCG00000003105" "ENSSSCG00000006090" "ENSSSCG00000022404" "ENSSSCG00000012275" "ENSSSCG0000
## [19] "ENSSSCG00000016745" "ENSSSCG00000017089" "ENSSSCG00000027659" "ENSSSCG00000015443" "ENSSSCG0000
## [25] "ENSSSCG00000012880" "ENSSSCG00000006925" "ENSSSCG00000014016" "ENSSSCG00000003085" "ENSSSCG0000
## [31] "ENSSSCG00000011968" "ENSSSCG00000007745" "ENSSSCG00000024622" "ENSSSCG00000003402" "ENSSSCG0000
## [37] "ENSSSCG00000025106" "ENSSSCG00000017635" "ENSSSCG00000003240" "ENSSSCG00000023080" "ENSSSCG0000
## [43] "ENSSSCG00000001780" "ENSSSCG00000025836" "ENSSSCG00000001418" "ENSSSCG00000013072" "ENSSSCG0000
## [49] "ENSSSCG00000021285" "ENSSSCG00000022474"
##
## $RNA$value
##                         value.var
## ENSSSCG00000011119  0.3189379494
## ENSSSCG00000026418  0.3097023326
## ENSSSCG00000011546 -0.3065501353
## ENSSSCG00000017092  0.2649259031
## ENSSSCG00000020871  0.2544068137
## ENSSSCG00000007431  0.2518168632
## ENSSSCG00000000038  0.2201308131
## ENSSSCG00000024536  0.2188622383
## ENSSSCG00000022482 -0.1972603718
## ENSSSCG00000021418  0.1875484860
## ENSSSCG00000025109  0.1827310613
## ENSSSCG00000000764 -0.1805732838
## ENSSSCG00000003105  0.1714658238
## ENSSSCG00000006090 -0.1680230103
## ENSSSCG00000022404  0.1554914742
## ENSSSCG00000012275  0.1349227172
## ENSSSCG00000010132  0.1294867120
## ENSSSCG00000012657  0.1183566452
## ENSSSCG00000016745  0.1107912312
## ENSSSCG00000017089  0.1106486088
## ENSSSCG00000027659  0.1086846347
## ENSSSCG00000015443  0.1078087221
## ENSSSCG00000022990 -0.1036326497
## ENSSSCG00000014338  0.1020670047
```

```
## ENSSSCG00000012880  0.0931649343
## ENSSSCG00000006925  0.0898529507
## ENSSSCG00000014016  0.0860933780
## ENSSSCG00000003085  0.0844405751
## ENSSSCG00000009512  0.0828968249
## ENSSSCG00000007664  0.0806195868
## ENSSSCG00000011968  0.0712316213
## ENSSSCG00000007745  0.0644456228
## ENSSSCG00000024622 -0.0579154567
## ENSSSCG00000003402  0.0499486018
## ENSSSCG00000006488  0.0455128122
## ENSSSCG00000027489  0.0441773982
## ENSSSCG00000025106  0.0425613002
## ENSSSCG00000017635  0.0415683910
## ENSSSCG00000003240 -0.0380635112
## ENSSSCG00000023080  0.0353577184
## ENSSSCG00000022970  0.0343074573
## ENSSSCG00000000142  0.0326019110
## ENSSSCG00000001780  0.0227409806
## ENSSSCG00000025836  0.0225125788
## ENSSSCG00000001418  0.0144276931
## ENSSSCG00000013072  0.0101236910
## ENSSSCG00000004063  0.0098616043
## ENSSSCG00000026269  0.0065395431
## ENSSSCG00000021285  0.0030601987
## ENSSSCG00000022474  0.0004122731
##
##
## $comp
## [1] 1
```

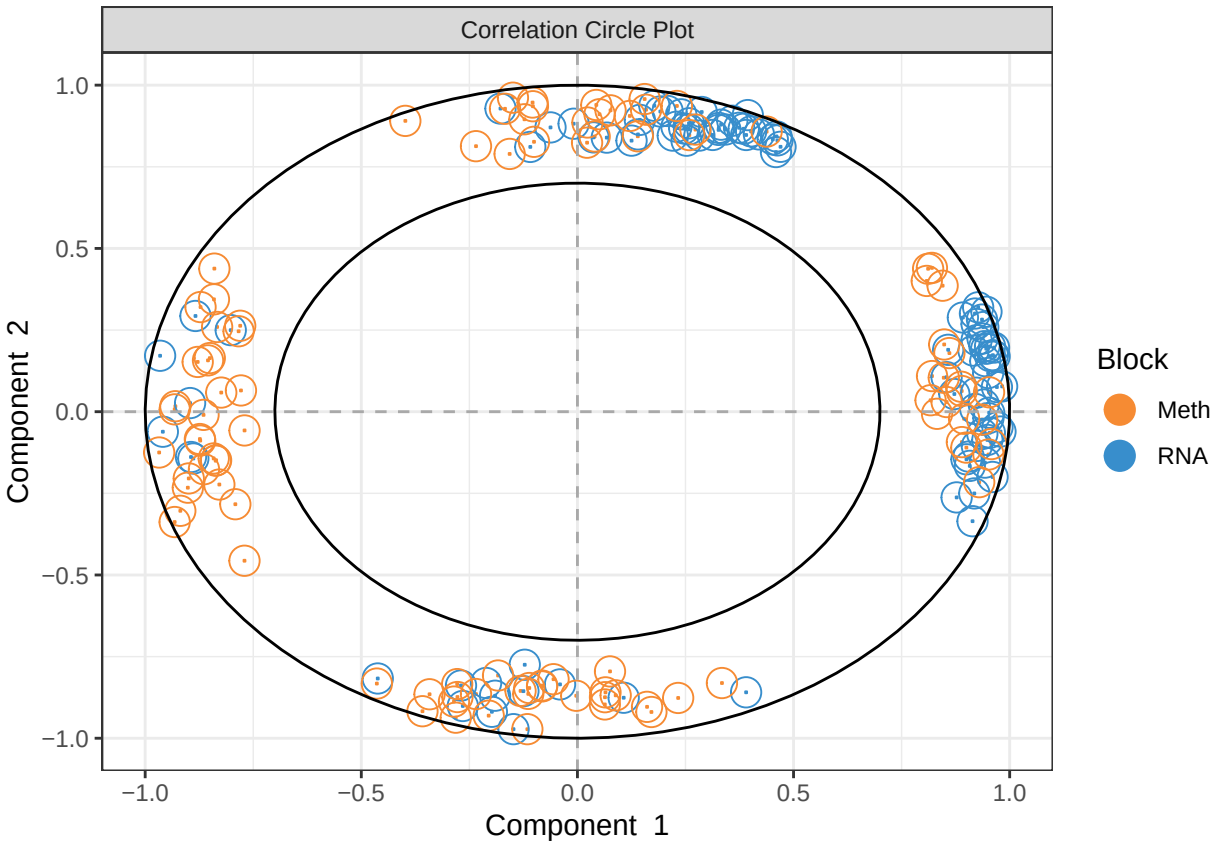
```
dim(var1$RNA$value)
```

```
## [1] 50  1
```

10.4 Correlation-circle variable plot

The correlation circle plot visualizes the relationships among the previously selected variables from both modalities. As our objective is to focus on the `comp=1`, each point represents a molecular feature projected onto the first latent component, where the distance from the center reflects its contribution to the model. Each colour represent the matrix block data, where orange and blue represents respectively Methylome and Transcriptome modalities. A cutoff of the first circle is set at `0.7`, indicating the points observed between `abs(0.7 - 1)` are the highly correlated variables. Variables located close to one another are positively correlated, whereas variables positioned in opposite directions are negatively correlated.

```
# Visualize correlations among selected variables across all blocks
# Helpful for understanding relationships between RNA and Methylation features
plotVar(diablo_model, var.names = F,rad.in = 0.7, legend = T )
```




```
# Uncomment this block if you're running old linux OS
# ragg::agg_png(filename = "plot_var_correlation.png", width = 3200, height = 1800, units = "px", res =
# plotVar(diablo_model, var.names = F,rad.in = 0.7, legend = T )
# dev.off()
```

10.5 Generate the DIABLO clustered image map

The clustered heatmap displays the RNA and methylation features selected on component 1 across the nine boars. The samples cluster according to their molecular profiles, with the three Medium individuals forming a distinct group, while the Low and High samples show more similar patterns. The columns also group selected RNA and methylation variables with comparable expression or methylation profiles. The individual differentiations between the Medium taint vs the Low and High matches with the results observed in `plotIndiv()`. Similarly, we are able to differentiate between the correlations observed among the data blocks and the sample groups.

```
###### Run this chunk for the Heatmap of selected features from a DIABLO
### expression or methylation values
# colours = color.GreenRed(50, alpha = 1)
# cim_res <- cimDiablo(diablo_model, comp = 1, color = colours)
# cim_res

# Uncomment this block if you're running old linux OS // old the plot isn't loading in R
colours = color.GreenRed(50, alpha = 1)
pdf("cim_diablo_heatmap.pdf", width = 10, height = 10)
```

```r
cim_res <- cimDiablo(diablo_model, comp = 1, color = colours)
```

```
##
## trimming values to [-3, 3] range for cim visualisation. See 'trim' arg in ?cimDiablo
```

```r
dev.off()
```

```
## pdf
##   2
```

```r
summary(cim_res)
```

```
##                Length Class      Mode
## mat            900    -none-     numeric
## row.names        9    -none-     character
## col.names      100    -none-     character
## row.sideColors   9    -none-     character
## col.sideColors 100    -none-     character
## rowInd           9    -none-     numeric
## ddr              2    dendrogram list
## colInd         100    -none-     numeric
## ddc              2    dendrogram list
```

```r
dim(cim_res$mat)
```

```
## [1]   9 100
```

```r
head(rownames(cim_res$mat))
```

```
## [1] "L1137" "L9312" "L1105" "M1113" "H9138" "H7107"
```

```r
head(colnames(cim_res$mat))
```

```
## [1] "ENSSSCG00000024622" "chr3_129945605"     "chr7_905201"        "ENSSSCG00000011546" "chr17_35677
```

10.6 Prepare heatmap annotations

This plot provides another representation of the selected variables, with clearer annotation of the feature names correlated with the phenotype outcomes.

```r
# Map row.sideColors to labels
row_color_to_label <- c("#388ECC" = "High", "#C2C2C2" = "Medium", "#F68B33" = "Low")
row_labels <- row_color_to_label[cim_res$row.sideColors]

row_annotation <- data.frame(Phenotypes = factor(row_labels, levels = c("Low", "Medium", "High")))
rownames(row_annotation) <- cim_res$row.names

# Map col.sideColors to labels
col_color_to_label <- c("#1F78B4" = "RNA", "#33A02C" = "Meth")
col_labels <- col_color_to_label[cim_res$col.sideColors]

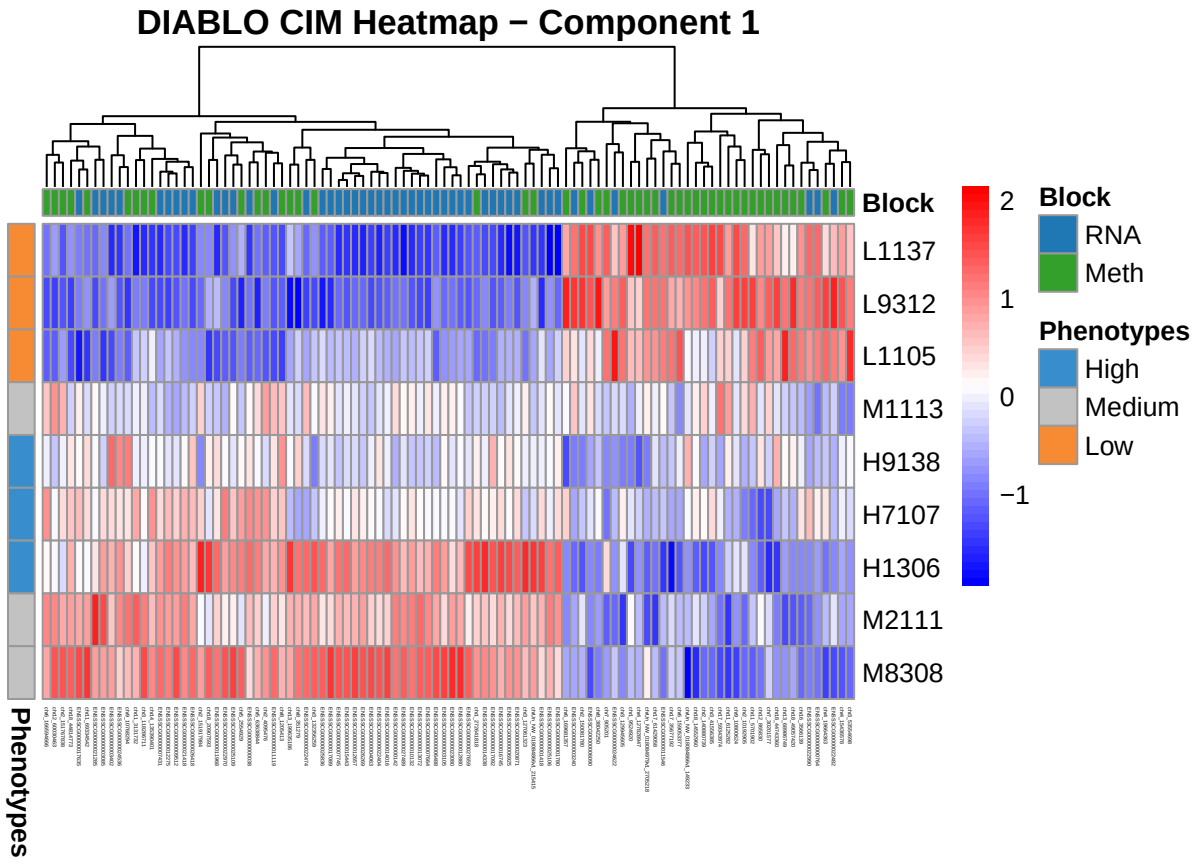
col_annotation <- data.frame(Block = factor(col_labels, levels = c("RNA", "Meth")))
rownames(col_annotation) <- cim_res$col.names

annotation_colors <- list(
  Block = c(
```

```
    RNA = "#1F78B4",
    Meth = "#33A02C"
  ), Phenotypes = c(High = "#388ECC", Medium = "#C2C2C2", Low = "#F68B33")
)

# There are 2 blocks RNA and Meth in columns, with feature IDS in the x-axis. There are 9 BT samples la
#The scale going from -2 to +2 is effect sizes (estimated weights from the model) estimated from DIABLO

pheatmap(cim_res$mat,
         cluster_rows = F,
         cluster_cols = T,
         show_rownames = TRUE,
         show_colnames = TRUE,
         annotation_row = row_annotation,
         annotation_col = col_annotation,
         annotation_colors = annotation_colors,
         color = colorRampPalette(c("blue", "white", "red"))(50),
         main = "DIABLO CIM Heatmap - Component 1",
         fontsize_col = 2,      # column names font size
         fontsize_row = 10,      # row names font size
         fontsize = 10)          # general font size for heatmap
```




```
# Uncomment this block if you're running old linux OS // old the plot isn't loading in R

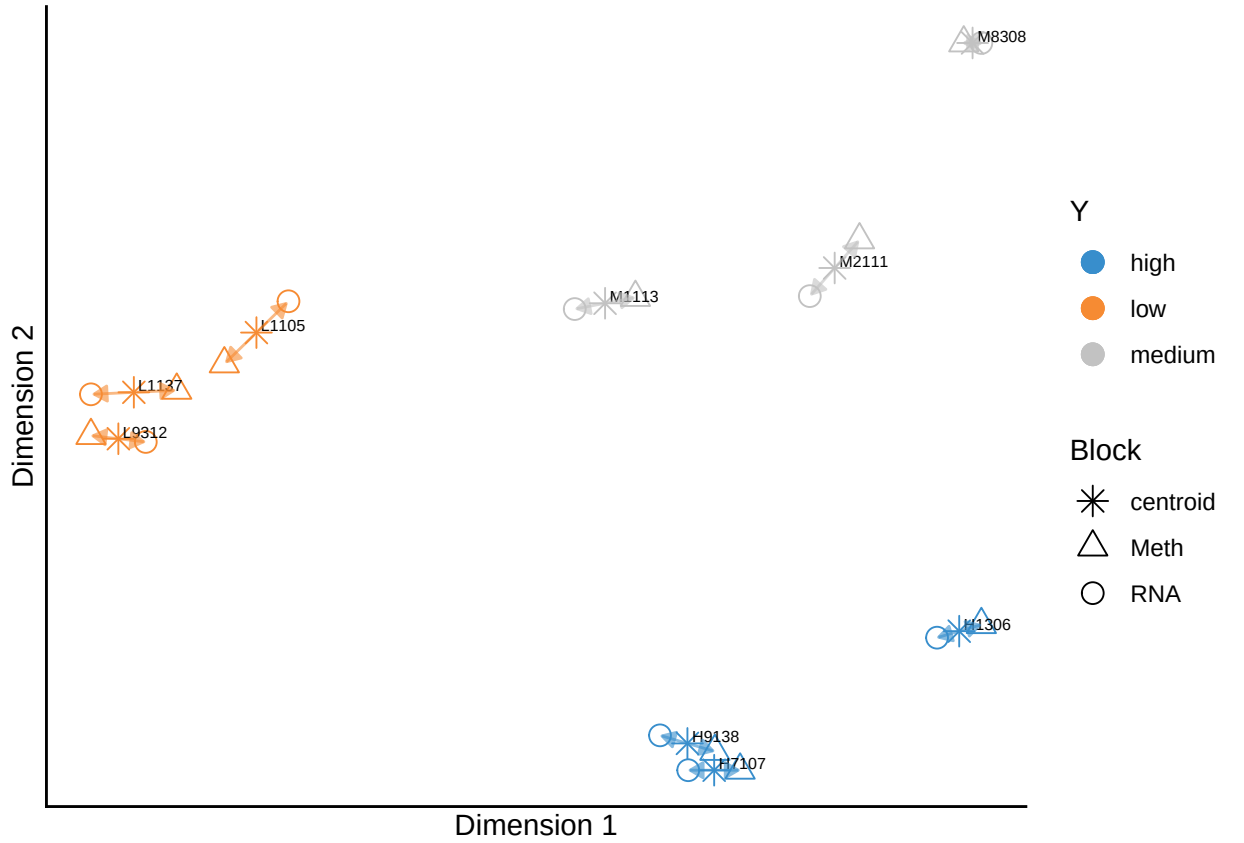
# ragg::agg_png(filename = "plot_cim_pheatmap_new.png", width = 3200, height = 2400, units = "px", res
# pheatmap(cim_res$mat,
```

```
#           cluster_rows = F,
#           cluster_cols = T,
#           show_rownames = TRUE,
#           show_colnames = TRUE,
#           annotation_row = row_annotation,
#           annotation_col = col_annotation,
#           annotation_colors = annotation_colors,
#           color = colorRampPalette(c("blue", "white", "red"))(50),
#           main = "DIABLO CIM Heatmap - Component 1",
#           fontsize_col = 2,      # column names font size
#           fontsize_row = 10,      # row names font size
#           fontsize = 10)          # general font size for heatmap
# dev.off()
```

10.7 Sample-arrow plot

In the arrow plot below, the starting point of each arrow represents the consensus position of a given boar across the RNA and methylation datasets, while the arrow tips indicate its block-specific position in each omics dataset. Each point represent the sample observed from an omic dataset, represented by the legend color, and the arrow tip This visualization therefore highlights the agreement between the two data blocks at the individual-sample level. Shorter arrows indicate stronger agreement between RNA and methylation measurements, whereas longer arrows indicate greater disagreement between the omics-specific representations of the same sample.

```
# Plot sample projections and agreement between omics blocks
# Arrows show how each sample's omics-specific and consensus positions differ
plotArrow(
  diablo_model,
  ind.names = TRUE,
  legend = TRUE,
  title = "DIABLO - Sample Arrows",
  cex = 0.9,  # Adjust label size
  pch.size = 3.5
  )
```

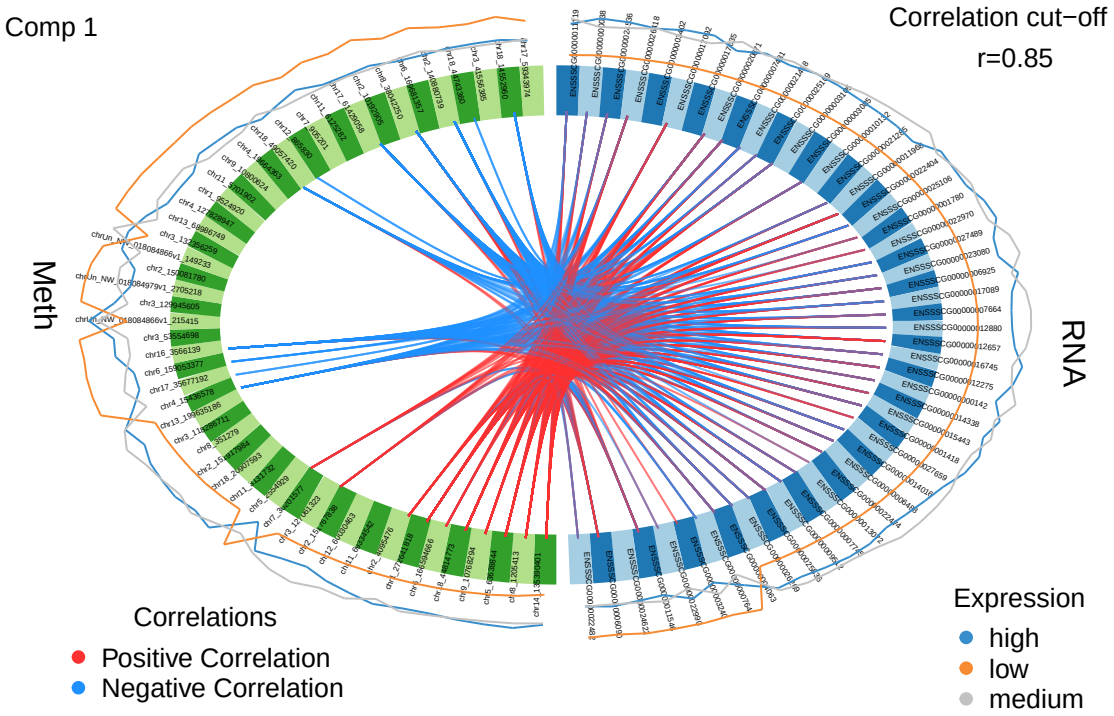



```
# Uncomment this block if you're running old linux OS // old the plot isn't loading in R
# ragg::agg_png(filename = "plot_arrow.png", width = 3200, height = 1800, units = "px", res = 300, back
# plotArrow(
#   diablo_model,
#   ind.names = TRUE,
#   legend = TRUE,
#   title = "DIABLO - Sample Arrows",
#   cex = 0.9,  # Adjust label size
#   pch.size = 3.5
#   )
# dev.off()
```

10.8 Circos plot

The circos plot represents the strongest correlations between the selected RNA and methylation variables on component 1, with the two omics blocks displayed on opposite sides of the circle. Red links indicate positive correlations, whereas blue links indicate negative correlations. The result is matching and summarizing all the previous heatmaps, loading contributions and correlation circle plots. In this plot, both positive and negative relationships are observed between the RNA and methylation features. The outer profiles show the relative expression or methylation patterns associated with the High, Low, and Medium phenotype groups. Only correlations with an absolute value above the selected threshold of 0.85 are displayed, so the interpretation is restricted to very strongly correlated features.

```
# Generate a circos plot from the DIABLO model to visualize correlations between features
circosPlot(diablo_model, comp = 1, line = TRUE, cutoff = 0.85)   ### Its good
```




```
# Uncomment this block if you're running old linux OS // old the plot isn't loading in R

# ragg::agg_png(filename = "plot_circos.png", width = 3200, height = 1800, units = "px", res = 300, bac
# circosPlot(diablo_model, comp = 1, line = TRUE, cutoff = 0.85)   ### Its good
# dev.off()
```

10.9.a Extract high-correlation RNA–methylation pairs

In this step, we will be extracting the pairs of omics information that seem to be correlated, with a cutoff higher than the absolute value of 0.85.

```
# Extract correlation matrix between RNA and Meth blocks, comp = 1
# Get the selected variables
rna_loadings <- diablo_model$loadings$RNA[, 1]
meth_loadings <- diablo_model$loadings$Meth[, 1]

# Set a loading threshold (e.g., abs > 0.1)
# Filter based on loadings (optional, or use all features)
top_rna <- names(rna_loadings[abs(rna_loadings) > 0.1])
top_meth <- names(meth_loadings[abs(meth_loadings) > 0.1])
```

```r
# Subset expression and methylation matrices
rna_mat <- diablo_model$X$RNA[, top_rna, drop = FALSE]
meth_mat <- diablo_model$X$Meth[, top_meth, drop = FALSE]

# Compute pairwise correlations
cor_matrix <- cor(rna_mat, meth_mat, use = "pairwise.complete.obs") #This builds a correlation matrix o

# Flatten the correlation matrix into long format
cor_long <- melt(cor_matrix, varnames = c("RNA", "Meth"), value.name = "correlation")

# Filter based on desired correlation cutoff
high_cor <- subset(cor_long, abs(correlation) >= 0.85)

# Sort by correlation strength
high_cor <- high_cor[order(-abs(high_cor$correlation)), ]

# Get matrix of high-correlation features only
heat_cor_mat <- cor_matrix[unique(high_cor$RNA), unique(high_cor$Meth)]

my_palette <- colorRampPalette(rev(brewer.pal(n = 11, name = "RdBu")))(1000)

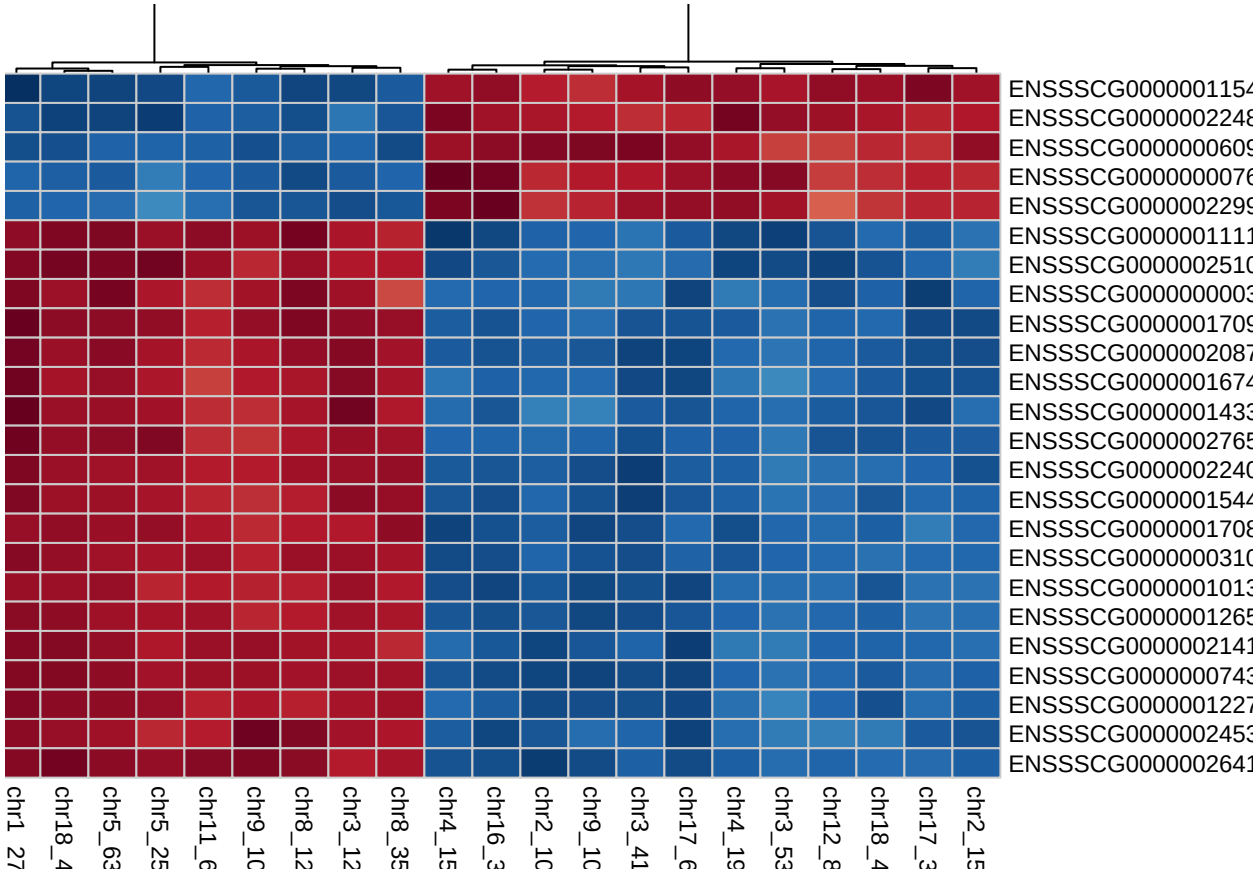
# Create the heatmap

heat_cor_mat <- as.matrix(heat_cor_mat)
```

10.9.b Correlation heatmap

After selecting the top variables that contributed to the first component, we have plotted them respectively to their high correlation value. This heatmap clearly indicates clusters of biomarkers that are negatively / positively correlated. An evaluation of these biomarkers are recommended at the genomic location for a better causality interpretation.

```r
pheatmap::pheatmap(
  heat_cor_mat,
  color = my_palette,
  cluster_rows = TRUE,
  cluster_cols = TRUE,
  show_rownames = TRUE,
  show_colnames = TRUE,
  fontsize = 10,
  fontsize_row = 9,
  fontsize_col = 9,
  cellwidth = 18,
  cellheight = 11,
  main = "High-Correlation RNA-Methylation Pairs",
  border_color = "grey80",
  legend_breaks = c(-1, -0.85, 0.85, 1),
  legend_labels = c("-1", "-0.85", "0.85", "1"),
  legend = TRUE)
```




```
  # Uncomment this block if you're running old linux OS // old the plot isn't loading in R

# ragg::agg_png(filename = "plot_correlation_top_loadings.png", width = 3200, height = 2400, units = "ps
# pheatmap::pheatmap(
#   heat_cor_mat,
#   color = my_palette,
#   cluster_rows = TRUE,
#   cluster_cols = TRUE,
#   show_rownames = TRUE,
#   show_colnames = TRUE,
#   fontsize = 10,
#   fontsize_row = 9,
#   fontsize_col = 9,
#   cellwidth = 18,
#   cellheight = 11,
#   main = "High-Correlation RNA-Methylation Pairs",
#   border_color = "grey80",
#   legend_breaks = c(-1, -0.85, 0.85, 1),
#   legend_labels = c("-1", "-0.85", "0.85", "1"),
#   legend = TRUE
# )
# dev.off()
```

10.10 RNA–methylation correlation network

The correlation network represents the strong associations identified between the selected RNA and methylation features. RNA variables are shown in red, while methylation sites are shown in blue, and each edge represents a correlation between the two omics blocks. The network contains 154 RNA–methylation pairs with correlations above 0.85, highlighting highly connected features that may contribute jointly to the separation of the boar taint phenotypes. The network can also be exported in `graphml` format for further exploration in `Cytoscape`.

```
##### Network analysis of the genes and CPGs
# Create edges from gene-Meth pairs
edges <- data.frame(
  from = high_cor$RNA,
  to = high_cor$Meth,
  weight = high_cor$correlation
)

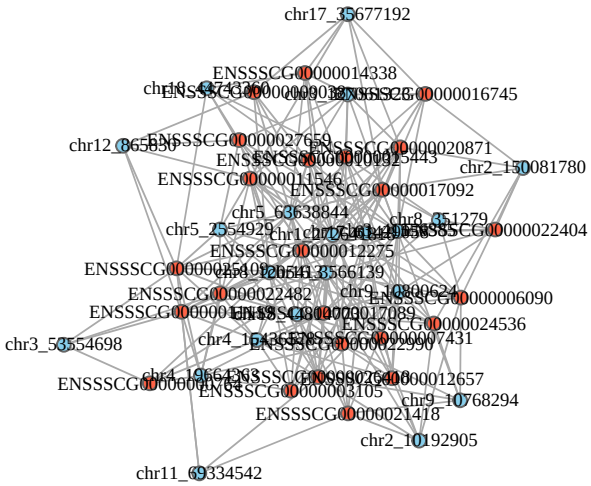
# Create graph from edge list
g <- graph_from_data_frame(edges, directed = FALSE)

# Define node types correctly (RNA vs Meth)
V(g)$type <- ifelse(V(g)$name %in% high_cor$RNA, "RNA", "Meth")

# Assign colors based on node type
V(g)$color <- ifelse(V(g)$type == "RNA", "tomato", "skyblue")  # tomato for RNA, skyblue for Meth

# Use absolute correlation values as edge weights
E(g)$weight <- abs(E(g)$weight)

# Plot the graph with weighted edges and node color based on type
plot(g,
     vertex.label.cex = 0.6,
     vertex.label.color = "black",
     vertex.size = 6,
     edge.width = E(g)$weight * 1,
     vertex.frame.color = "gray40",
     main = "Correlation Network: RNA - Methylation Features")
```




```
# Uncomment this block if you're running old linux OS // old the plot isn't loading in R
# ragg::agg_png(filename = "plot_correlation_network.png", width = 3200, height = 1800, units = "px", r
# plot(g,
#      vertex.label.cex = 0.6,
#      vertex.label.color = "black",
#      vertex.size = 6,
#      edge.width = E(g)$weight * 1,
#      vertex.frame.color = "gray40",
#      main = "Correlation Network: RNA   Methylation Features")
# dev.off()

dim(high_cor[high_cor$correlation > 0.85,]) # 154 pairwise comparison
```

```
## [1] 99  3
```

```
#### Save the networks info for the Cytosacpe
write_graph(g, file = "gene_methylation_network.graphml", format = "graphml")
```

---

11. Performance evaluation

The predictive performance of the DIABLO model was evaluated using the previously computed using `leave-one-out method`, and ROC analyses for the RNA and methylation blocks. For the methylation block, component 1 showed perfect discrimination for the Medium group, with an AUC of 1.00, while the Low and High groups reached AUC values of 0.7778 and 0.7222, respectively. These results should be interpreted cautiously because the dataset contains only nine observations, with three individuals per phenotype group.

In the same way, the observation on the RNA block also resulted a perfect discrimination of the Medium group, while the rest have lower value. Subsequently, this small sample size strongly limits the stability and generalizability of the estimated AUC values and may lead to overly bias interpretation. In general, multiomics integration requires quite numerous observation to overcome the curse of dimensionality.

```
#############################################################################################################

# run repeated CV performance evaluation
perf.diablo
```

```
##
## Call:
##  perf.sgccda(object = diablo_model, validation = "loo", nrepeat = 10, progressBar = TRUE, BPPARAM = c
##
##  Main numerical outputs:
##  --------------------
##  Error rate (overall or BER) for each component and for each distance: see object$error.rate
##  Error rate per class, for each component and for each distance: see object$error.rate.class
##  Prediction values for each component: see object$predict
##  Classification of each sample, for each component and for each distance: see object$class
##  Stable features on each component: see object$features$stable
##  AUC values: see object$auc if auc = TRUE
##
##  Visualisation Functions:
##  --------------------
##  plot
```

```
perf.diablo$MajorityVote.error.rate
```

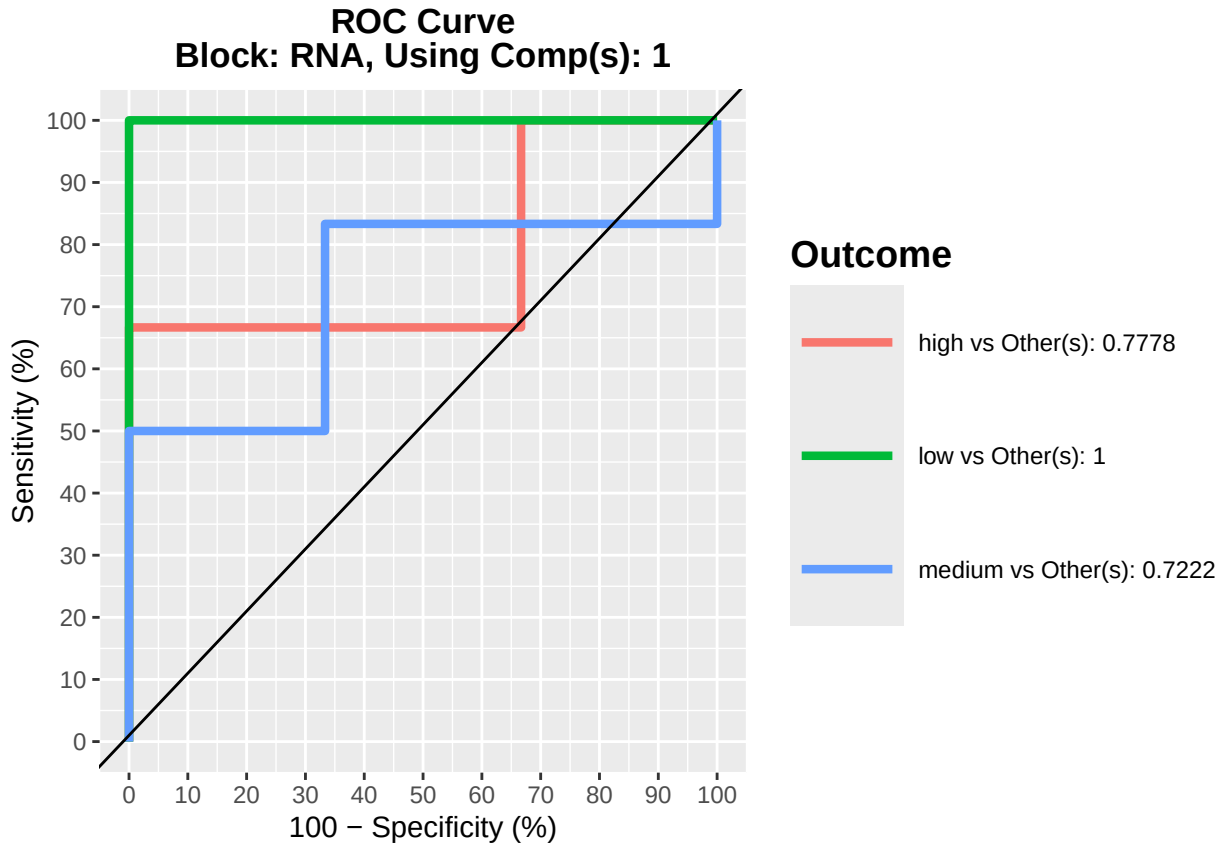
```
## $max.dist
##             comp1 comp2 comp3 comp4 comp5
## high            1     1     1     1     1
## low             1     1     1     1     1
## medium          1     1     1     1     1
## Overall.ER      1     1     1     1     1
## Overall.BER     1     1     1     1     1
##
## $centroids.dist
##                 comp1 comp2 comp3 comp4 comp5
## high        0.6666667     1     1     1     1
## low         1.0000000     1     1     1     1
## medium      1.0000000     1     1     1     1
## Overall.ER  0.8888889     1     1     1     1
## Overall.BER 0.8888889     1     1     1     1
##
## $mahalanobis.dist
##                 comp1     comp2 comp3 comp4 comp5
## high        0.6666667 1.0000000     1     1     1
## low         1.0000000 1.0000000     1     1     1
## medium      1.0000000 0.6666667     1     1     1
## Overall.ER  0.8888889 0.8888889     1     1     1
## Overall.BER 0.8888889 0.8888889     1     1     1
```

```
perf.diablo$WeightedVote.error.rate
```

```
## $max.dist
```

```
##             comp1 comp2 comp3 comp4 comp5
## high            1     1     1     1     1
## low             1     1     1     1     1
## medium          1     1     1     1     1
## Overall.ER      1     1     1     1     1
## Overall.BER     1     1     1     1     1
##
## $centroids.dist
##                 comp1 comp2 comp3 comp4 comp5
## high        0.3333333     1     1     1     1
## low         0.0000000     1     1     1     1
## medium      1.0000000     1     1     1     1
## Overall.ER  0.4444444     1     1     1     1
## Overall.BER 0.4444444     1     1     1     1
##
## $mahalanobis.dist
##                 comp1     comp2 comp3 comp4     comp5
## high        0.3333333 1.0000000     1     1 1.0000000
## low         0.0000000 1.0000000     1     1 0.6666667
## medium      1.0000000 0.6666667     1     1 1.0000000
## Overall.ER  0.4444444 0.8888889     1     1 0.8888889
## Overall.BER 0.4444444 0.8888889     1     1 0.8888889
```

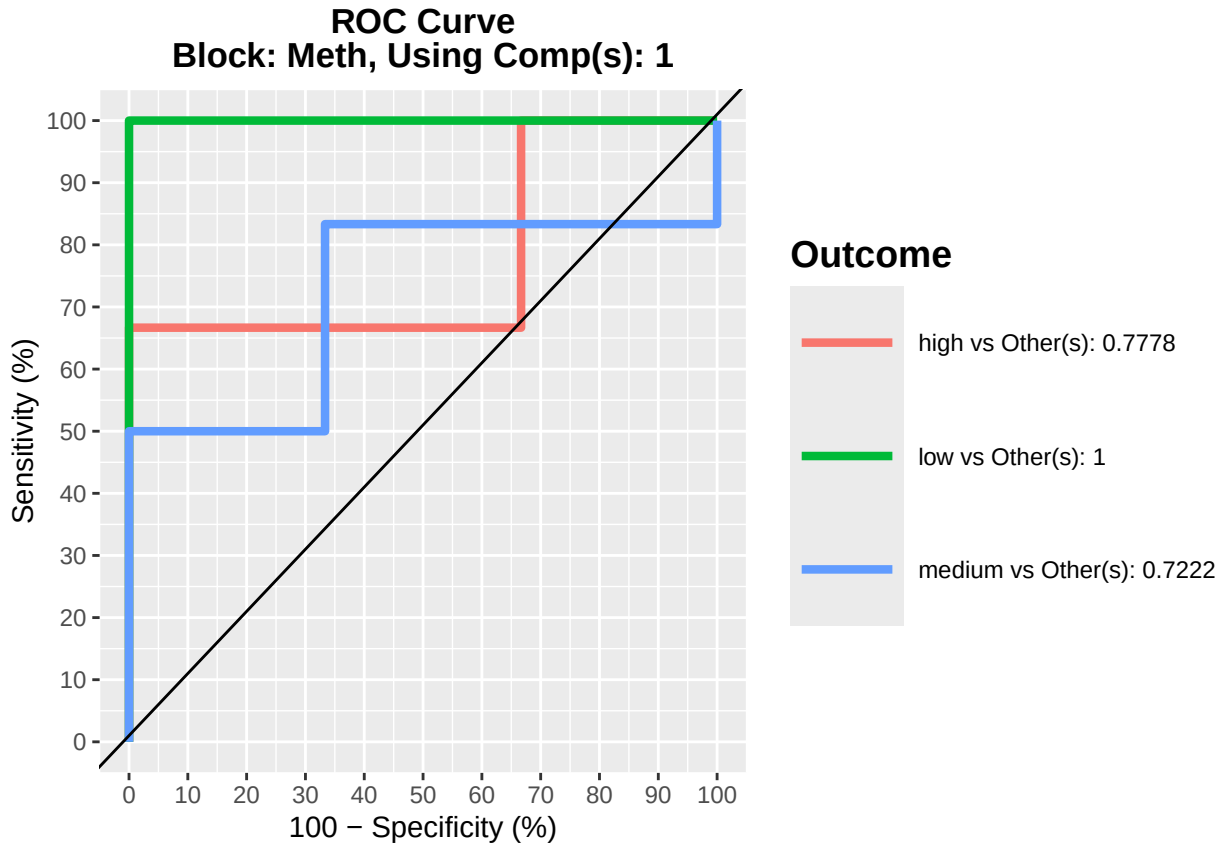
```
auc.splsda = auroc(diablo_model, roc.block = "RNA",
                   roc.comp = 1, print = TRUE, plot = TRUE)
```




```
## $RNA
```

```
## $RNA$comp1
##                       AUC p-value
## high vs Other(s)   0.7778 0.19670
## low vs Other(s)    1.0000 0.02014
## medium vs Other(s) 0.7222 0.30170
##
## $RNA$comp2
##                     AUC p-value
## high vs Other(s)      1 0.02014
## low vs Other(s)       1 0.02014
## medium vs Other(s)    1 0.02014
##
##
## $Meth
## $Meth$comp1
##                       AUC p-value
## high vs Other(s)   0.7778 0.19670
## low vs Other(s)    1.0000 0.02014
## medium vs Other(s) 0.7222 0.30170
##
## $Meth$comp2
##                     AUC p-value
## high vs Other(s)      1 0.02014
## low vs Other(s)       1 0.02014
## medium vs Other(s)    1 0.02014
```

```
auc.splsda = auroc(diablo_model, roc.block = "Meth",
                   roc.comp = 1, print = TRUE, plot = TRUE)
```




```
## $RNA
## $RNA$comp1
##                         AUC p-value
## high vs Other(s)   0.7778 0.19670
## low vs Other(s)    1.0000 0.02014
## medium vs Other(s) 0.7222 0.30170
##
## $RNA$comp2
##                     AUC p-value
## high vs Other(s)      1 0.02014
## low vs Other(s)       1 0.02014
## medium vs Other(s)    1 0.02014
##
##
## $Meth
## $Meth$comp1
##                         AUC p-value
## high vs Other(s)   0.7778 0.19670
## low vs Other(s)    1.0000 0.02014
## medium vs Other(s) 0.7222 0.30170
##
## $Meth$comp2
##                     AUC p-value
## high vs Other(s)      1 0.02014
## low vs Other(s)       1 0.02014
## medium vs Other(s)    1 0.02014
```

```
# Uncomment this block if you're running old linux OS // old the plot isn't loading in R
# ragg::agg_png(filename = "plot_AUC_RNA.png", width = 3200, height = 1800, units = "px", res = 300, ba
# auc.splsda = auroc(diablo_model, roc.block = "RNA",
#                    roc.comp = 1, print = TRUE, plot = TRUE)
# dev.off()
#
# ragg::agg_png(filename = "plot_AUC_Meth.png", width = 3200, height = 1800, units = "px", res = 300, b
# auc.splsda = auroc(diablo_model, roc.block = "Meth",
#                    roc.comp = 1, print = TRUE, plot = TRUE)
# dev.off()
```

---

12. Optional top-feature network analysis

The following code was commented out in the supplied script and remains commented out here.

```
# ############### Only on Top features Network analysis  ############################
#
# Extract loadings
load_rna  <- diablo_model$loadings$RNA[, 1]
load_meth <- diablo_model$loadings$Meth[, 1]

# Select top 10 features based on absolute loading
top_rna <- names(sort(abs(load_rna), decreasing = TRUE))[1:10]
top_meth <- names(sort(abs(load_meth), decreasing = TRUE))[1:10]

# Subset omics data to top 10 features
omics_top <- list(RNA = omics_list_fixed$RNA[, top_rna],
                  Meth = omics_list_fixed$Meth[, top_meth])

diablo_small <- block.splsda(
  X = omics_top,
  Y = Y,
  ncomp = 1,
  # only 1 component needed for network
  keepX = list(RNA = 10, Meth = 10),
  design = matrix(c(0, 1, 1, 0), ncol = 2)  # simple design
)
```

```
## Design matrix has changed to include Y; each block will be
##             linked to Y.
```

```
network_graph <- network(
  diablo_small,
  comp = list(RNA = 1, Meth = 1),
  blocks = c("RNA", "Meth"),
  color.node = c("tomato", "skyblue"),
  shape.node = c("circle", "rectangle"),
  cutoff = 0.3,
  save = "jpeg"
)

# Uncomment this block if you're running old linux OS // old the plot isn't loading in R
```

```
# ragg::agg_png(
#   filename = "plot_small_network.png",
#   width = 3200,
#   height = 1800,
#   units = "px",
#   res = 300,
#   background = "white"
# )
#
# network_graph <- network(
#   diablo_small,
#   comp = list(RNA = 1, Meth = 1),
#   blocks = c("RNA", "Meth"),
#   color.node = c("tomato", "skyblue"),
#   shape.node = c("circle", "rectangle"),
#   cutoff = 0.3,
#   save = "jpeg"
# )
#
# igraph::V(network_graph)$label.family <- "sans"
#
# plot(
#   network_graph,
#   vertex.label.family = "sans"
# )
#
# dev.off()
```

---

13. Acknowledgement

Special thanks to the PhD student Pratik Pandurang for sharing the original data, and some parts of the script. Major changes in the script was done by Adam El Idrissi. Special thanks to the core team of Mixomics for building this amazing integration software.

For any further questions, feel free to contact me in the following contacts: Email: adam.elidrissi@unicatt.it - tnewnis2@gmail.com